



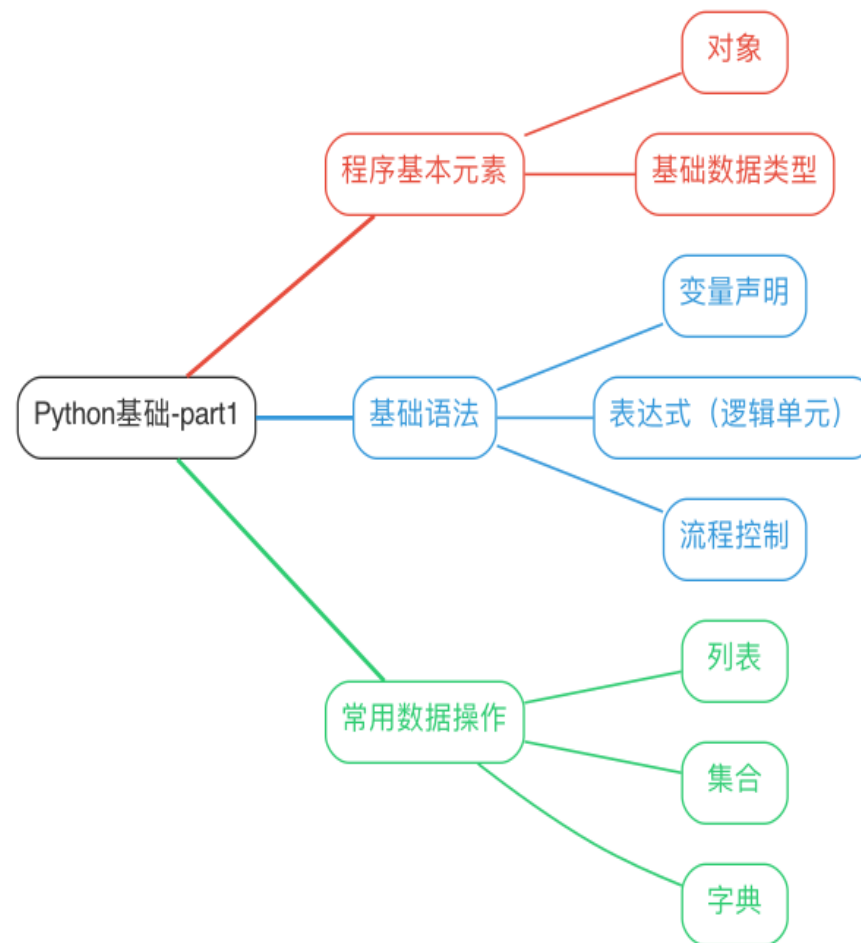
《实用Python编程》

第2讲：Python基础-part1

实用Python编程 教研组



提纲





Python程序中的基本元素

- 让我们从一个例子开始
 - 先试着理解一下这段代码是做什么的

user_input和val都是变量,
记录用户输入的内容

val%2 是算术表达式
val%2 == 0是关系表达式

if-else选择性执行指定语句

```
user_input = input('请输入一个整数:')  
val = int(user_input)  
  
if val%2 == 0:  
    print(f'你输入的{user_input}是一个双数')  
else:  
    print(f'你输入的{user_input}是一个单数')
```

请输入一个整数: ↑↓ for history. Search history with c-↑/c-↓

input是函数, 接受用户从键
盘输入的内容

int是函数, 将字符串变为整数

print也是函数, 将指定内容
输出到屏幕上

变量、函数、表达式、流程控制等构成了Python程序的基本元素, 这些都是对象 (object)



一切皆对象

- 对象(object)是Python程序中数据及其操作的载体
 - 属性 (Attribute)：对象内部存储的数据
 - 方法 (Method)：对象可执行的操作
 - 标识符 (Identity)：对象的存储地址

示例代码 2.1: 列表对象、集合对象的创建与简单操作。注意 # 开头的文字是代码注释，无需输入

```
1 lst = [] #创建一个名为lst的空列表对象
2 lst.append(1) # 往列表中添加一个整数1
3 print(len(lst)) # 通过len函数获得lst中的元素个数（即1）并通过print函数输出到屏幕上
4 lst.append(2) # 往列表中再添加一个整数2
5 print(len(lst)) # 输出 2
6 print(id(lst)) # 输出lst所指列表对象的标识符，形如140535170940736
7
8 st = set() #创建一个名为st的空集合对象
9 st.add(1) #往集合中添加一个整数1
10 print(len(st)) # 获取st中的元素个数（即1）并输出到屏幕上
11 st.add(2) #往集合中再添加一个整数2
12 print(len(st)) # 输出 2
13 print(type(st)) # 输出 <class 'set'>
```

在Notebook中
运行示例代码
并观察结果



基础数据类型

- 数据类型 (data type): 具有相同性质的对象的集合

表 2.1: 常用数据类型

数据类型	功能	示例
对象内容不可变:		
int	整型, 用来表示整数值	5, -3, 1000
float	浮点型, 用来表示小数	3.14, -0.001, 2.0
str	字符串, 字符序列	用双引号或者单引号包围, 如 "hello", 'Python'。空字符串用 "" 或 '' 表示, 长度为 0
tuple	元组, 有序的不可变集合	用圆括号表示, 如三元组 (1, 2, 3)。元组一旦创建, 其内容不可修改
对象内容可变:		
list	列表, 有序的可变集合	用方括号表示, 如 [1, 2, 3]。创建之后可增加、删除、修改其元素
set	集合, 无序的可变集合	通过 set() 创建, 如 set([1, 2, 3]) 将列表转为集合
dict	字典, 键值对 (key-value pair) 集合	用花括号表示, 如 {"name": "Alice", "age": 25}。表示空字典



基础数据类型

- 使用内容不可变的数据类型可以避免误操作
 - 可用三元组表示出生日期，这样创建之后再尝试修改，代码就会报错

示例代码 2.2 元组内容不可变

```
1 birth = (2020, 10, 1) # 用以记录出生年月日的三元组
2 birth[0] = 2021 # 尝试将出生年份改为2021
3 # 程序报错
4 # TypeError: 'tuple' object does not support item assignment
```



基础语法

- 变量 (variable) : 对特定对象的命名

示例代码 2.3: 声明变量

```
1 name = "Alice" # 声明一个字符串变量
2 age = 22 # 声明一个整数变量
3 name = [] # 再让name指向一个空的列表对象
4 word2count = {} # 声明一个指向空字典对象的变量
5 data_filename = 'C:/Users/xiron/pycode/data/imdb_labelled.txt' # 声明一个字符串变量
```

- 命名规则

- 由英文字母、数字、下划线组成, 但不能以数字开头
- 不能使用Python关键字 / 保留字
- 区分大小写 / 大小写敏感

```
import keyword
print(keyword.kwlist)
```

Python3支持包括中文在内的Unicode 字符作为标识符, 中文或特殊符号也能作为变量名。但不推荐。



课堂练习-2-1-1

尝试声明不同数据类型的变量，体会

- 数值（整数、浮点数）、字符串（英文、暂时不考虑中文）
- 变量的命名规则（一开始就养成好的编程习惯）
- 大小写敏感（但不推荐）
- 同一变量指向不同数据类型（但不推荐）
- 不能使用关键字（尝试在Notebook中执行 `False = 2`）

基础语法

- 表达式 (expression)：由常数、变量、运算符及函数调用等组成的语法结构，是Python程序中最基本的逻辑单元

表达式的常见功能

- 赋值
- 数值计算
- 关系比较
- 逻辑运算
- 函数调用
- 成员判断
- 身份判断
- ...

```
1 import math
2
3 x, y = 3, 2 # 赋值, 令x为3, y为2
4 z = x ** y # 数值表达式: 幂运算
5 print(x > y) # 关系表达式
6 print(x < y) # 关系表达式
7 z = x%2 == 0 # 表达式嵌套: 关系表达式中内置了一个算术表达式
8 z = "偶数" if x%2==0 else "奇数"
9 z = math.sqrt(x) # 函数调用, 计算平方根
10 print(x in [1, 2, 3]) # 成员判断
11 print(x not in [1, 2, 3]) # 成员判断
```

基础语法

- 运算优先级

- 当表达式中包含多个运算符时，运算的执行顺序由运算符优先级决定
 - 方括号[]: 索引 / 切片
 - 点号.: 访问属性
 - 算术运算: 乘方 (幂) > 乘、除 > 加、减
 - 比较运算: 各种比较运算同级
 - 赋值运算: 总在最后执行
- 对于同一级的运算符，则按自左向右顺序进行计算

记忆可能会出错。好的编程习惯是在表达式中用括号明确运算优先级

```
1 x = 1+3*4/2 # x的值为 7
2 x = (1+3)*4/2 # 利用括号让加法先执行, x的值为 8
3 y = [1, 2, 3]
4 z = y[0] + y[1] # 先访问元素再相加, z的值为 3
5 x, y, z = True, False, False
6 w = x or y and z # 先执行 y and z 获得 False, 再执行 x or False 获得 True
7 w = x and y and z # 先执行 x and y 获得 False, 无需再执行 and, 返回 False
```



课堂练习-2-1-2

- 假设一个城市的人口 P 和年份存在如下的关系:

$$P = 250,000e^{0.012t}$$

$t = 0$ 表示2017年, $t = 1$ 表示2018年, ...

问题

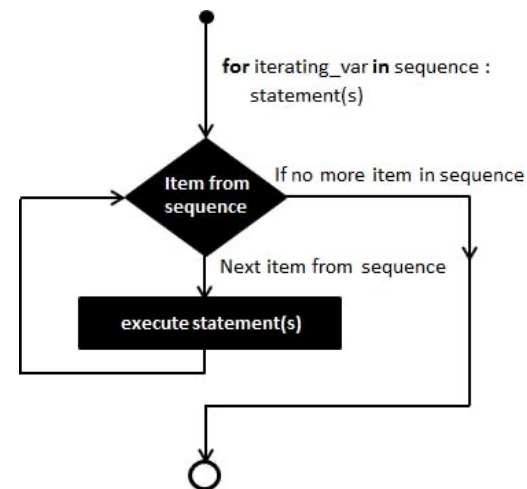
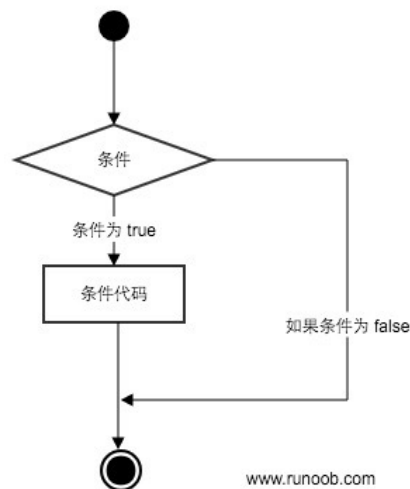
- ① 2020年的人口数是多少?
-

提示

- ① google 一下 python 如何计算指数函数 e^x
 - ② 问一下大模型: 怎么用 python 计算指数函数
-

基础语法

- 流程控制 (Flow Control) : 程序运行时每条代码的执行顺序
 - 顺序结构
 - 条件分支 (if-else)
 - 循环结构 (for / while)





流程控制

- if-else条件分支

分数定级

```
score = 85
# if-else从此开始
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
# if-else至此结束
print(grade) # 输出评级
```

语法模板

```
if 条件1: # 第1个分支
    # 当条件1为True时执行
    # 代码块1
elif 条件2: # 第2个分支, 可选
    # 条件2为True时执行
    # 代码块2
elif 条件3: # 第3个分支, 可选
    # 条件3为True时执行
    # 代码块3
else:      # 最后一个分支, 可选
    # 其他情况执行
    # 代码块4
```



流程控制

- 使用for循环遍历一个列表

```
1 data = [("李四", 92), ("张三", 86), ("王五", 82)]
2 for x in data:
3     name = x[0]
4     score = x[1]
5     print(name, score) # 逐行输出名字和分数
6
7 for (name, score) in data:
8     print(name, score) # 逐行输出名字和分数
9
10 for rank, (name, score) in enumerate(data, start=1):
11     print(name, score, rank) # 逐行输出名字、分数和排名
12
13 # 通过遍历找出最高分
14 best_score = -100 # 用于记录遍历过程中找到的最高分
15 best = -1 # 用于记录最高分学生在列表中的位置
16 for i, x in enumerate(data):
17     if x[1] > best_score: # 找到更高分, 更新
18         best_score = x[1]
19         best = i
20 print("最高分: ", data[best]) # 最高分: ("李四", 92)
```



流程控制

- 使用内置的`range`函数产生循环序列

```
1 data = [("李四", 92), ("张三", 86), ("王五", 82)]
2 n = len(data) # 获取列表长度, 即列表中元素的个数 (本例中为3)
3 for i in range(n):
4     name = data[i][0]
5     score = data[i][1]
6     print(name, score) # 逐行输出名字和分数
```

```
李四 92
张三 86
王五 82
```



课堂练习2-1-3

- 比较`range(10)`, `range(1,10)`, `range(1,10,2)`的循环范围



流程控制

更简洁的for循环使用

- 列表推导式 (List Comprehension)
- 适用场景：可单行完成的数据处理

```
a = [i*5 for i in range(1,11)] # 生成5的倍数序列 [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
b = [i for i in range(10) if i % 2 == 0] # 生成10以内的双数序列 [0, 2, 4, 6, 8]
```

流程控制

- 当循环次数无法预先指定时，使用while循环

尝试从data中找出所有分数大于85的记录

```
1 data = [("李四", 92), ("王五", 82), ("张三", 86)]
2
3 i = 0 # 位置索引, 从0开始
4 while (i < len(data)) and (data[i][1] > 85): # 组合式条件
5     print(data[i]) # 输出学生信息
6     i = i + 1 # 更新索引, 用于访问列表中的下一个元素
```

('李四', 92)

未输出所有满足条件的记录，代码有逻辑错误（Bug）。怎么解决？

循环中断

当我们需要改变循环的正常执行流程时，可用break和continue语句

- 只能在循环体中使用
- 执行到break语句时，立即结束当前循环
- 执行到continue语句时，直接进入下一次循环
- 逻辑上，要求与if-else条件语句配合使用（为什么？）

```
1 data = [("李四", 92), ("张三", 86), ("王五", 82)]
2
3 # 仅输出列表中第一个分数大于85的元素
4 for (name,score) in data:
5     if score > 85:
6         print(name, score)
7         break # 循环结束
8 print('-----')
9 # 输出列表中所有分数大于85的元素
10 for (name,score) in data:
11     if score <= 85:
12         continue # 跳过循环体的剩余部分代码
13     print(name, score)
```

李四 92

李四 92

张三 86



课堂练习2-1-4

- 在课堂练习2-1-2的基础上

- 假设一个城市的人口 P 和年份存在如下的关系:

$$P = 250,000e^{0.012t}$$

$t = 0$ 表示2017年, $t = 1$ 表示2018年, ...

问题

- ① 2020年的人口数是多少?
- ② 什么时间人口数达到400,000?



流程控制

• try-except异常处理

带有异常处理机制的除法程序

```
try:
    num1 = int(input("请输入一个整数作为被除数: "))
    num2 = int(input("请输入一个整数作为除数: "))
    result = num1 / num2
    print(f"{num1} / {num2} = {result}")
except ValueError:
    print("【错误提示】 请输入有效的整数")
except ZeroDivisionError:
    print("【错误提示】 除数不能为零")
except Exception as e:
    print(f"【未知错误】 {e}")
finally:
    print("*** 程序结束 ***")
```

语法模板

```
try:
    # 【尝试区】：把可能报错的代码放在这里
except:
    # 【处理区】：如果上面报错了，则执行这里的代码
[finally:]
    # 【收尾区】 无论是否报错，最后都会执行
```



常用数据操作

- 三种最常用的“容器类”数据类型
 - 列表 (list)、集合 (set)、字典 (dict)

列表

- 用于顺序存储数据
- 示例 `numbers = [1, 2, 3, 4, 5]`
- 典型场景：存储一组同类型数据

```
1 a = [] # 空列表
2 numbers = [1, 2, 3, 4, 5] # 包含数字的列表
3 names = ["张三", "李四", "王五"] # 包含字符串的列表
4 # 使用乘法
5 zeros = [0] * 5 # 创建包含5个0的列表: [0, 0, 0, 0, 0]
6 # 使用range()函数
7 numbers = list(range(5)) # [0, 1, 2, 3, 4]
8 # 使用列表推导式
9 steps = [i*2 for i in range(5)] # 创建一个步长为2的整数序列 [0, 2, 4, 6, 8]
```

访问列表

- 读取列表中的元素
 - 通过索引（从0开始）访问单个元素
 - 通过切片（左闭右开）访问一组元素

```
1 # 使用索引访问单个元素
2 my_list = ['a', 'b', 'c', 'd', 'e']
3 print(my_list[0]) # 'a' - 第一个元素
4 print(my_list[1]) # 'b' - 第二个元素
5 print(my_list[2]) # 'c' - 第三个元素
6 print(my_list[-1]) # 'e' - 倒数第一个元素
7 print(my_list[-2]) # 'd' - 倒数第二个元素
8 print(my_list[-3]) # 'c' - 倒数第三个元素
9 # 使用切片访问子序列
10 print(my_list[1:3]) # ['b', 'c'] - 索引1到2(不包括3)
11 print(my_list[:3]) # ['a', 'b', 'c'] - 从头至索引2
12 print(my_list[3:]) # ['c', 'd', 'e'] - 从索引3到末尾
13 print(my_list[::2]) # ['a', 'c', 'e'] - 间隔取元素
14 print(my_list[::-1]) # ['e', 'd', 'c', 'b', 'a'] - 反转列表
```

操作列表

• 增、删、改、查

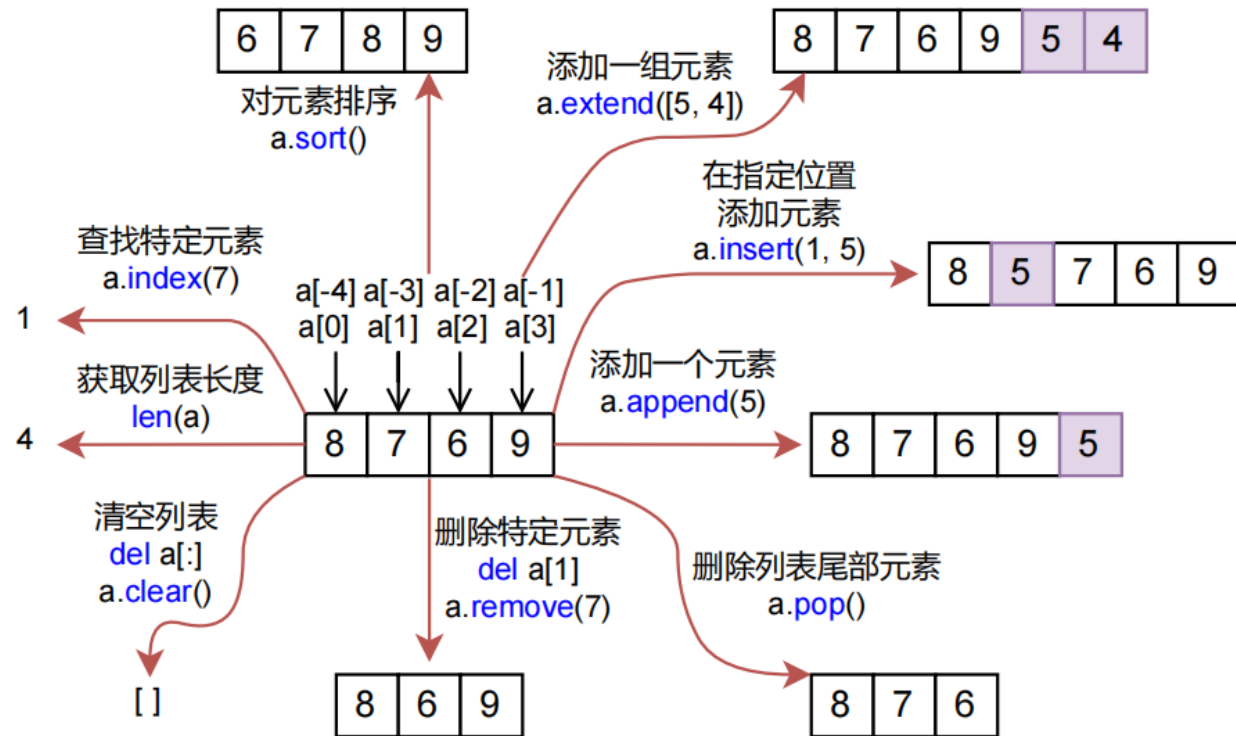


图 2.1: 列表常用操作示例。图中的 `a` 为指向列表 `[8, 7, 6, 9]` 的变量

集合

- 集合存储数据的特点
 - 元素的存储地址是无序的
 - 不能通过索引访问特定元素
 - 集合中元素取值不能重复
- 示例 `s=set([1, 2, 3, 3])` # {1, 2, 3}注意元素3已被去重
- 典型场景：去重、快速判断元素是否存在

集合的日常操作

- 创建集合
- 添加、删除元素
- 成员测试

```
1 # 创建集合
2 s = set() # 空集合
3 s = set([1, 2, 3, 4]) # 从列表创建 {1, 2, 3, 4}
4 s = set([1, 2, 3, 3]) # {1, 2, 3} 注意元素3已被去重
5 s = {1, 2, 3, 4} # 直接使用花括号
6
7 # 添加元素
8 s.add(5) # 添加一个元素, 集合变为{1, 2, 3, 4, 5}
9 s.update([6, 7]) # 添加多个元素, 集合变为{1, 2, 3, 4, 5, 6, 7}
10
11 # 删除元素
12 s.remove(7) # 如果元素不存在会报告KeyError
13 s.discard(10) # 安全移除, 不存在也不会报错
14
15 # 成员测试
16 print(1 in s) # 输出 True
17 print(7 in s) # 输出 False
18
19 # 获取集合大小, 即集合中元素的个数
20 len(s) # 6
21
22 # 清空集合
23 s.clear()
24
25 # 将集合变为列表
26 s = {1, 2, 3, 4, 3} # 集合会自动去重, 产生集合 {1, 2, 3, 4}
27 t = list(s) # 产生列表 [1, 2, 3, 4]
```

集合的日常操作

- 用for循环遍历集合中的元素

相同元素只能
出现一次

```
1 basic_colors = {'红色', '蓝色', '绿色', '黄色', '黑色', '白色', '红色'}  
2 for color in basic_colors:  
3     print(color)
```

蓝色
绿色
黑色
白色
红色
黄色



字典

- 存储特点
 - 逻辑结构：键-值对(Key-Value Pair)的集合
 - 通过“键”访问“值”，“键”唯一
 - 示例
 - 取快递时，取件码就是键，而快递件就是值
 - 典型场景：描述对象属性，计数

字典的日常操作

- 创建字典
- 元素访问、遍历
- 添加、修改、删除

```
1 # 创建字典
2 d = {} # 空字典
3 d = dict() # 空字典
4 d1 = {'name': 'Alice', 'age': 25} # 直接初始化
5 d2 = dict(name='Alice', age=25) # 通过关键字参数初始化
6 d3 = dict([('name', 'Alice'), ('age', 25)]) # 从键值对序列创建
7 d4 = dict.fromkeys(['a', 'b', 'c'], 0) # 从键列表创建默认值字典 {'a': 0, 'b': 0, 'c': 0}
8
9 # 以键 (key) 为索引来访问元素
10 print(d1['name']) # 'Alice' (键不存在会报KeyError)
11 print(d1.get('age')) # 25 (安全访问, 不存在返回None)
12 print(d1.get('gender', 'unknown')) # 不存在则返回指定值'unknown'
13
14 # 检查键 (key) 是否存在
15 print('name' in d2) # 输出 True
16 print('Alice' in d2) # 输出 False
17
18 # 添加/修改元素
19 d1['gender'] = 'female' # 添加新键值对
20 d1['age'] = 26 # 修改已有键的值
21
22 # 删除元素
23 del d1['age'] # 删除键值对
24 value = d1.pop('name') # 删除并返回值
25 d1.clear() # 清空字典
26
27 # 通过items()遍历字典
28 for key, value in d4.items():
29     print(f"{key}:{value}") # 逐行输出 a:0, b:0, c:0
30
31 # 通过keys()遍历字典
32 for key in d4.keys():
33     print(f"{key}:{d4[key]}") # 逐行输出 a:0, b:0, c
```

特殊字典类型

- 内置模块collections中的Counter

```
1 from collections import Counter
2 a = ['a', 'b', 'b', 'c', 'c', 'c']
3 c = Counter()
4 for i,x in enumerate(a):
5     c[x] = c[x] + 1
6     print(f'iter-{i}', c.most_common())
```

```
iter-0 [('a', 1)]
iter-1 [('a', 1), ('b', 1)]
iter-2 [('b', 2), ('a', 1)]
iter-3 [('b', 2), ('a', 1), ('c', 1)]
iter-4 [('b', 2), ('c', 2), ('a', 1)]
iter-5 [('c', 3), ('b', 2), ('a', 1)]
```



列表、集合、字典的特点总结与对比

表 2.3: 容器类数据类型对比。时间复杂度 $O(1)$ 表示操作耗时相对固定，与数据量无关， $O(n)$ 意味着操作耗时与数据量成正比

数据类型	核心特性	元素操作的时间复杂度	适用场景
列表	元素可重复、以索引取值	查找 / 插入 / 删除 $O(n)$	数据收集、顺序处理
集合	元素唯一、不支持索引访问	查找 / 插入 / 删除 $O(1)$	去重、成员检测、集合运算
字典	键-值对、键唯一、以键取值	查找 / 插入 / 删除 $O(1)$	快速查询、配置管理、数据聚合

下列哪些语句一定或可能使用了列表/集合/字典对象？

```
len(a)
x in a
del a[1]
a.append("Tom")
a.add("Tom")
```